

西南科技大学

2024-2025-1 学期《网络安全技术》

实验 1 实验报告

实验名称：虚拟蜜罐实验

实验类型：验证性实验

指导教师：李凌

专业班级：

姓名：

学号：

小组组号：第 组

实验地点：个人计算机。

实验日期：2024 年 09 月 18 日

实验成绩：_____

一、实验目的

1. 深入理解蜜罐技术的工作原理

通过搭建 Honeyd 虚拟蜜罐环境，学习蜜罐技术在网络安全中的应用。通过 Honeyd 模拟虚拟主机，探索蜜罐如何作为诱捕工具，吸引和监控潜在攻击者的行为，分析其在安全防护体系中的作用。

2. 掌握 Honeyd 的安装与配置方法

通过安装配置 Honeyd，了解其依赖库（如 Libevent、Libdnet、Libpcap 和 Arpd 工具）的作用及安装顺序，确保 Honeyd 能正常运行。这一过程帮助掌握软件安装和故障排查的技能，特别是与网络安全相关的环境搭建。

3. 学习 Honeyd 配置文件的编写

配置文件是 Honeyd 实现多样化服务模拟的关键。通过配置 Honeyd 的蜜罐主机模板、系统特性、端口开放策略、服务模拟等，熟悉配置文件的编写和调试技巧，以实现对不同操作系统、服务的模拟，提高对网络协议和服务的理解。

4. 理解低交互蜜罐的特点与适用场景

Honeyd 是一种低交互蜜罐，不提供真实的业务服务，而是通过简化的服务响应来模拟真实主机。通过实验，体会低交互蜜罐的优缺点，理解它如何避免与攻击者深度交互，从而降低蜜罐被识破的风险，同时也认识到它在网络流量分析、恶意行为捕获中的局限性。

5. 探索如何使用蜜罐收集攻击数据

利用 Honeyd 模拟多种操作系统和服务，吸引外部的恶意扫描和攻击行为，学习如何通过蜜罐记录网络连接、协议交互和攻击模式等信息，为进一步的安全分析提供数据支持。

6. 验证 Honeyd 在不同协议和端口上的模拟能力

在 Honeyd 配置中开放特定端口（如 TCP 80、22、23、53），分别模拟 Web、SSH、Telnet 等常用服务，并使用不同的客户端工具（如浏览器、Xshell）和网络扫描工具（如 nmap）来验证这些服务的可用性，观察并分析其响应行为，从而深入理解 Honeyd 对网络服务的模拟能力。

二、实验设计

实验环境：

宿主机 Ubuntu 的 ip 为：192.168.204.165

```
alspp@alspp-virtual-machine:~/桌面$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.204.165 netmask 255.255.255.0 broadcast 192.168.204.255
    inet6 fe80::28e7:79c7:52f2:dfb2 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:7f:ee:0a txqueuelen 1000 (Ethernet)
    RX packets 16529 bytes 22917944 (22.9 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3796 bytes 254792 (254.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 193 bytes 17732 (17.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 193 bytes 17732 (17.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

测试机 Win10 的 ip 为: 192.168.204.171

```
C:\Users\26032>ipconfig

Windows IP 配置

以太网适配器 Ethernet0:

    连接特定的 DNS 后缀 . . . . . : localdomain
    本地链接 IPv6 地址. . . . . : fe80::3959:230f:374f:4ef5%2
    IPv4 地址 . . . . . : 192.168.204.171
    子网掩码 . . . . . : 255.255.255.0
    默认网关. . . . . : 192.168.204.2

以太网适配器 蓝牙网络连接:

    媒体状态 . . . . . : 媒体已断开连接
    连接特定的 DNS 后缀 . . . . . :
```

处于同一局域网下，能相互 ping 通

```
C:\Users\26032>ping 192.168.204.165

正在 Ping 192.168.204.165 具有 32 字节的数据:
来自 192.168.204.165 的回复: 字节=32 时间=1ms TTL=64
来自 192.168.204.165 的回复: 字节=32 时间=1ms TTL=64
来自 192.168.204.165 的回复: 字节=32 时间<1ms TTL=64
来自 192.168.204.165 的回复: 字节=32 时间=1ms TTL=64

192.168.204.165 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 0ms, 最长 = 1ms, 平均 = 0ms
```

三、实验过程（包含实验结果）

环境准备

Honeyd 的安装和配置

Honeyd 软件依赖于下面几个库及 arpd 工具：

(1) Libevent：是一个非同步事件通知的函数库。

通过使用 libevent，开发者能够设定某些事件发生时所运行的函数，能够取代以往程序所使用的循环检查；

(2) Libdnet：是一个提供了跨平台的网络相关 API 的函数库，包含 arp 缓存，路由表查询。IP 包及物理帧的传输等。

(3) Libpcap：是一个数据包捕获（Packet Sniffing）的函数库，大多数网络软件都以它为基础；

(4) Arpd 工具：arpd 执行在与 honeyd 同样的系统上。是 honeyd 众多协作工具中最重要的一个。Arpd 工作时监视局域网内的流量。并通过查看 honeyd 系统的 ARP 表推断其他系统的活动与否。

启动 honeyd 前有时必须先启动 arpd

启动 arpd 有 2 种

A. arpd IP

B. %arpd IP(%表示 arpd 的路径)由于系统本身有个 arpd，有时须要指定自己安装的那个 arpd 的路径。

Honeyd 安装 (linux)：

进入文件资料目录下打开 shell。

(注意：首先安装 libnet、libevent、libpcap 然后安装 arpd，最后安装 Honeyd)

Libnet：

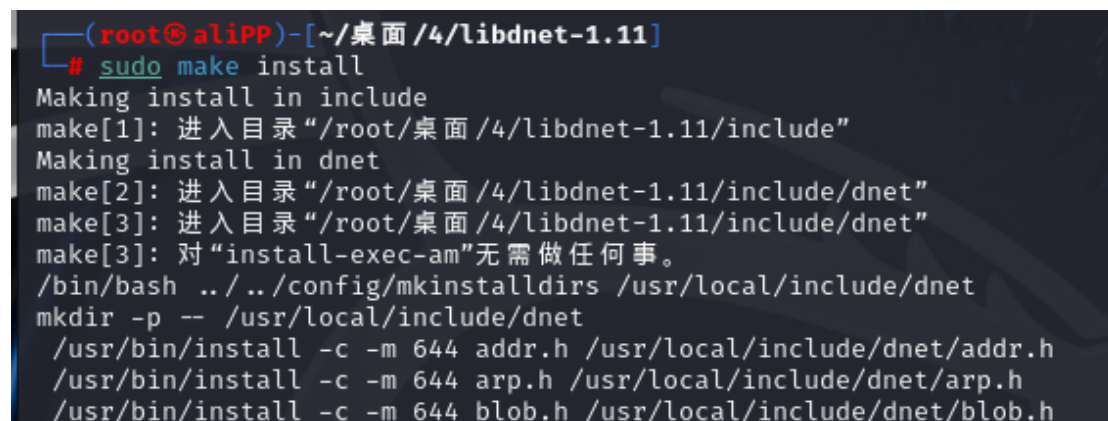
tar -zxvf libdnet-1.11.tgz

cd libdnet-1.11/

./configure

make

sudo make install



```
(root@alipp)~[~/桌面/4/libdnet-1.11]
# sudo make install
Making install in include
make[1]: 进入目录 "/root/桌面/4/libdnet-1.11/include"
Making install in dnet
make[2]: 进入目录 "/root/桌面/4/libdnet-1.11/include/dnet"
make[3]: 进入目录 "/root/桌面/4/libdnet-1.11/include/dnet"
make[3]: 对 "install-exec-am" 无需做任何事。
/bin/bash ../../config/mkinstalldirs /usr/local/include/dnet
mkdir -p -- /usr/local/include/dnet
/usr/bin/install -c -m 644 addr.h /usr/local/include/dnet/addr.h
/usr/bin/install -c -m 644 arp.h /usr/local/include/dnet/arp.h
/usr/bin/install -c -m 644 blob.h /usr/local/include/dnet/blob.h
```

Libevent：

用 tar -zxvf libevent-1.4.14b-stable.tar.gz 解压缩

用 cd libevent-1.4.14b-stable 进入文件夹

用 ./configure 检测目标特征

用 make 进行编译

用高权限的 make install 进行安装

```
(root@aliPP)-[~/桌面/4/libevent-1.4.14b-stable]
# sudo make install
make install-recursive
make[1]: 进入目录 "/root/桌面/4/libevent-1.4.14b-stable"
Making install in .
make[2]: 进入目录 "/root/桌面/4/libevent-1.4.14b-stable"
make[3]: 进入目录 "/root/桌面/4/libevent-1.4.14b-stable"
test -z "/usr/local/bin" || /usr/bin/mkdir -p "/usr/local/bin"
/usr/bin/install -c event_rpcgen.py '/usr/local/bin'
test -z "/usr/local/lib" || /usr/bin/mkdir -p "/usr/local/lib"
/bin/bash ./libtool --mode=install /usr/bin/install -c libevent.la libevent_core.la libevent_extra.la '/usr/local/lib'
libtool: install: /usr/bin/install -c .libs/libevent-1.4.so.2.2.0 /usr/local/lib/libevent-1.4.so.2.2.0
libtool: install: (cd /usr/local/lib && { ln -s -f libevent-1.4.so.2.2.0 libevent-1.4.so.2 || { rm -f libevent-1.4.so.2 && ln -s libevent-1.4.so.2.2.0 libevent-1.4.so.2; }; })
libtool: install: (cd /usr/local/lib && { ln -s -f libevent-1.4.so.2.2.0 libevent.so || { rm -f libevent.so && ln -s libevent-1.4.so.2.2.0 libevent.so; }; })
libtool: install: /usr/bin/install -c .libs/libevent.lai /usr/local/lib/libevent.lai
```

Libpcap:

tar -zxvf libpcap-1.3.0.tar.gz

cd libpcap-1.3.0

./configure

make

*提示错误: make: yacc: 没有那个文件或目录

```
gcc -fpic -I. -DHAVE_CONFIG_H -D_U="__attribute__
./bpf_dump.c
./runlex.sh lex -os scanner.c scanner.l
mv scanner.c scanner.c.bottom
cat scanner.c.top scanner.c.bottom > scanner.c
yacc -d grammar.y
make: yacc: 没有那个文件或目录
make: *** [Makefile:461: grammar.c] 错误 127
```

apt-get install bison (安装 yacc)

```
(root@aliPP)-[~/桌面/4/libpcap-1.7.3]
# apt-get install bison
正在读取软件包列表 ... 完成
正在分析软件包的依赖关系树 ... 完成
正在读取状态信息 ... 完成
建议安装：
  bison-doc
下列【新】软件包将被安装：
  bison
升级了 0 个软件包，新安装了 1 个软件包，要卸载 0 个软件包，有 3 个软件包未被升级。
需要下载 1,175 kB 的归档。
解压缩后会消耗 3,191 kB 的额外空间。
0% [执行中]
```

make

sudo make install

```
(root@aliPP)-[~/桌面/4/libpcap-1.7.3]
# sudo make install
VER=`cat ./VERSION`; \
MAJOR_VER=`sed 's/\([0-9][0-9]*\)\\.*/\1/' ./VERSION`; \
gcc -shared -Wl,-soname,libpcap.so.$MAJOR_VER \
-o libpcap.so.$VER pcap-linux.o pcap-usb-linux.o pcap-can-linux.o pcap-ne
tfilter-linux.o fad-getad.o pcap.o inet.o gencode.o optimize.o nametoaddr.o e
therent.o savefile.o sf-pcap.o sf-pcap-ng.o pcap-common.o bpf_image.o bpf_dum
p.o scanner.o grammar.o bpf_filter.o version.o
[ -d /usr/local/lib ] || \
    (mkdir -p /usr/local/lib; chmod 755 /usr/local/lib)
VER=`cat ./VERSION`; \
MAJOR_VER=`sed 's/\([0-9][0-9]*\)\\.*/\1/' ./VERSION`; \
/usr/bin/install -c libpcap.so.$VER /usr/local/lib/libpcap.so.$VER; \
ln -sf libpcap.so.$VER /usr/local/lib/libpcap.so.$MAJOR_VER; \
ln -sf libpcap.so.$MAJOR_VER /usr/local/lib/libpcap.so
#
# Most platforms have separate suffixes for shared and
# archive libraries, so we install both.
#
[ -d /usr/local/lib ] || \
```

arpd:

tar arpd-0.2.tar.gz

cd arpd-0.2/

./configure

Make

编译报错。上网查询得到以下的解决方法:

```
(root@aliPP)-[~/桌面/4/arpd]
# make
gcc -DHAVE_CONFIG_H -I. -I. -I. -I/usr/local/include -I/usr/local/include -I/
usr/local/include -I/usr/local/include -c arpd.c
arpd.c: In function 'arpd_send':
arpd.c:268:47: error: expected ')' before string constant
268 |         syslog(LOG_DEBUG, __FUNCTION__ ": who-has %s tell %s"
    |                                     ^
    |                                     )
    |                                     ^
arpd.c: In function 'arpd_lookup':
arpd.c:285:47: error: expected ')' before string constant
285 |         syslog(LOG_DEBUG, __FUNCTION__ ": %s at %s",
    |                                     ^
    |                                     )
    |                                     ^
arpd.c:294:47: error: expected ')' before string constant
294 |         syslog(LOG_DEBUG, __FUNCTION__ ": no entry for %s",
    |                                     ^
    |                                     )
    |                                     ^
arpd.c:297:47: error: expected ')' before string constant
297 |         syslog(LOG_DEBUG, __FUNCTION__ ": %s at %s",
    |                                     ^
    |                                     )
    |                                     ^
arpd.c: In function 'arpd_recv_cb':
arpd.c:426:55: error: expected ')' before string constant
426 |         syslog(LOG_DEBUG, __FUNCTION__ ": %s at %s",
    |                                     ^
    |                                     )
    |                                     ^
```

在 arpd/arpd.c 文件里加入 `#define __FUNCTION__ ""`

```
#undef TIMEOUT_INITIALIZED

#include <event.h>
#include <dnet.h>
#include "tree.h"

#define ARPD_MAX_ACTIVE      600
#define ARPD_MAX_INACTIVE   300

#define PIDFILE               "/var/run/arpd.pid"
#define __FUNCTION__ ""
struct arp_req {
    struct addr      pa;
    int              cnt;
```

sudo make install

```
(root@aliPP)-[~/桌面/4/arpd]
# sudo make install
make[1]: 进入目录 "/root/桌面/4/arpd"
/bin/sh ./mkinstalldirs /usr/local/sbin
/usr/bin/install -c arpd /usr/local/sbin/arpd
make install-man8
make[2]: 进入目录 "/root/桌面/4/arpd"
/bin/sh ./mkinstalldirs /usr/local/man/man8
/usr/bin/install -c -m 644 ./arpd.8 /usr/local/man/man8/arpd.8
make[2]: 离开目录 "/root/桌面/4/arpd"
make[1]: 离开目录 "/root/桌面/4/arpd"
```

honeyd 的安装:

tar -zxvf honeyd-1.5c.tar.gz

cd honeyd-1.5c

./configure

*提示错误: 需安装 libedit 或 libreadline

```
checking for dlopen in -ldl... yes
checking for libpcap... yes
checking for pcap_get_selectable_fd in -lpcap... yes
checking for dnet-config... /usr/local/bin/dnet-config
checking whether libdnet is a libdumbnet... no
checking for libevent... yes
checking for event_priority_init in -levent... yes
checking for pcre-config... no
checking for libedit... no
checking for libreadline... no
configure: error: need either libedit or libreadline; install one of them
```

apt-get install libedit-dev (安装 libedit)


```
(root@aliPP)-[~/桌面/4/honeyd-1.5c]
# apt-get install libedit-dev
正在读取软件包列表 ... 完成
正在分析软件包的依赖关系树 ... 完成
正在读取状态信息 ... 完成
将会同时安装下列软件：
  libbsd-dev libmd-dev
下列【新】软件包将被安装：
  libbsd-dev libedit-dev libmd-dev
升级了 0 个软件包，新安装了 3 个软件包，要卸载 0 个软件包，有 3 个软件包未被
升级。
需要下载 427 kB 的归档。
解压缩后会消耗 1,512 kB 的额外空间。
您希望继续执行吗？ [Y/n] Y
获取：1 http://kali.download/kali kali-rolling/main amd64 libmd-dev amd64 1.1.
0-2 [54.9 kB]
获取：2 http://mirrors.neusoft.edu.cn/kali kali-rolling/main amd64 libbsd-dev
amd64 0.12.2-1 [258 kB]
获取：3 http://mirrors.neusoft.edu.cn/kali kali-rolling/main amd64 libedit-dev
amd64 3.1-20240808-1 [114 kB]
```

./configure

*提示错误：需安装 zlib 库

```
checking for err... yes
checking for strsep... yes
checking for strlcpy... yes
checking for strlcat... yes
checking for getopt_long... yes
checking for SHA1Update... no
checking for msg_accrights field in struct msghdr... no
checking for sun_len in socketaddr_un... no
checking for msg_control field in struct msghdr... no
checking for timeradd in sys/time.h... yes
checking for isblank in ctype.h... yes
checking for working addr_cmp in libdnet... configure: error: you need to in
tall a more recent version of libdnet
```

启动 honeyd 时出现报错“libdnet.1: can't open sharedobjectfile”，

locate libdnet

cp /usr/local/lib/libdnet.1.0.1 /usr/local/lib/libdnet.so.1.0.1

/sbin/ldconfig

Updated

Kali 更新最新版 libdnet 无效，无法成功安装 honeyd。更换为虚拟机为 Ubuntu，重复前面的安装。（文件夹名严禁含中文）

Honey 安装成功：输入 honeyd

```
root@alspp-virtual-machine:/home/alspp/桌面/honeyd/honeyd-1.5c# updatedb
root@alspp-virtual-machine:/home/alspp/桌面/honeyd/honeyd-1.5c# honeyd
Honeyd V1.5c Copyright (c) 2002-2007 Niels Provos
honeyd[145028]: started with
Warning: Impossible SI range in Class fingerprint "IBM OS/400 V4R2M0"
Warning: Impossible SI range in Class fingerprint "Microsoft Windows NT 4.0 SP3"
honeyd[145028]: listening promiscuously on ens33: (arp or ip proto 47 or (udp an
d src port 67 and dst port 68) or (ip )) and not ether src 00:0c:29:7f:ee:0a
Honeyd starting as background process
root@alspp-virtual-machine:/home/alspp/桌面/honeyd/honeyd-1.5c#
```


实验过程

参考配置：

1.模板配置：

配置文件为 Honeyd 的虚拟蜜罐提供模板。每个模板模拟一个系统，可用于多个实例。默认模板定义了未定义行为的处理。推荐顺序：缺省模板 -> 模板 1 -> 模板 2 等。

2.模板结构：

每个模板包含以下参数：

操作系统特征（模拟 IP 堆栈）

绑定虚拟 IP 地址或地址段

ICMP、TCP、UDP 响应行为

系统变量（如 UPTIME、DropRATE、UID 等）

3.主要命令示例：

创建模板：Create <模板名>

个性化设置：SET <模板名> PERSONALITY “<OS 名称>”

绑定 IP：BIND <IP 地址> <模板名>

端口定义：

缺省响应：SET DEFAULT <模板名> <协议> ACTION <状态>

特定端口行为：ADD <模板名> <协议> PORT <端口号> <状态>

Honeyd 安装完成后，将在/usr/local/share/Honeyd 目录下存放其配置、指纹、脚本等数据文件。该目录下的文件 config.sample，需将其重命名为 honeyd.conf

```
alspp@alspp-virtual-machine:/$ cd /usr/local/share/
alspp@alspp-virtual-machine:/usr/local/share$ cd honeyd/
alspp@alspp-virtual-machine:/usr/local/share/honeyd$ ls
config.ethernet  nmap.assoc  pf.os  webserver
config.sample    nmap.prints  README  xprobe2.conf
alspp@alspp-virtual-machine:/usr/local/share/honeyd$ mv config.sample honeyd.conf
mv: cannot move 'config.sample' to 'honeyd.conf': Permission denied
alspp@alspp-virtual-machine:/usr/local/share/honeyd$ sudo mv config.sample honeyd.conf
[sudo] password for alspp:
alspp@alspp-virtual-machine:/usr/local/share/honeyd$ sudo mv config.sample honeyd.conf
mv: cannot stat 'config.sample': No such file or directory
alspp@alspp-virtual-machine:/usr/local/share/honeyd$ ls
config.ethernet  nmap.assoc  pf.os  webserver
honeyd.conf      nmap.prints  README  xprobe2.conf
alspp@alspp-virtual-machine:/usr/local/share/honeyd$
```

并按照规定要求进行配置，

```
alspp@alspp-virtual-machine:/usr/local/share/honeyd$ sudo vim honeyd.conf
```

其配置文件中的内容大致修改如下：

```
# Example of a simple host template and its binding
```

```
create windows
```

```
set windows personality "Linux 2.4.7 (X86)"
```

```
set windows default tcp action reset
```

```
set windows default udp action reset
```

```
set windows default icmp action reset
add windows tcp port 80 "/home/alspp/桌面/honeyd/honeyd-1.5c/scripts/web.sh"
add windows tcp port 22 "/home/alspp/桌面/honeyd/honeyd-1.5c/scripts/test.sh"
add windows tcp port 23 "/home/alspp/桌面/honeyd/honeyd-1.5c/scripts/router-telnet.pl"
add windows udp port 53 open
bind 192.168.204.122 windows
```

在此配置文件中，Honeyd 用来模拟一个虚拟主机，命名为 windows，配置了多个服务以响应指定端口。

指定该主机的操作系统指纹为 Linux 2.4.7 (X86)

设置未定义的 TCP 端口默认响应为 reset，未定义的 UDP 端口同样响应 reset，ICMP 请求也返回 reset

将 TCP 80 端口（HTTP 服务端口）绑定到脚本 web.sh。当虚拟主机收到端口 80 上的连接请求时，将运行此脚本。

将 TCP 23 端口（Telnet 服务端口）绑定到脚本 router-telnet.pl。在端口 23 上的连接请求将触发该脚本。

将 UDP 53 端口设置为 open，即直接开放响应。通常用于模拟 DNS 服务端口。

绑定 IP 地址

```
# Example of a simple host template and its binding
create windows
set windows personality "Linux 2.4.7 (X86)"
set windows default tcp action reset
set windows default udp action reset
set windows default icmp action reset
add windows tcp port 80 "/home/alspp/桌面/honeyd/honeyd-1.5c/scripts/web.sh"
add windows tcp port 22 "/home/alspp/桌面/honeyd/honeyd-1.5c/scripts/test.sh"
add windows tcp port 23 "/home/alspp/桌面/honeyd/honeyd-1.5c/scripts/router-telnet.pl"
add windows udp port 53 open
bind 192.168.204.122 windows
```

开启监控过程：

■ 启动 Arpd arpd 192.168.204.122

（启动 Arpd 侦听工具，在接收虚拟 IP 的 MAC 地址并用于查询时，将使用宿主机的 MAC 地址做出 ARP 应答）

● 启动 Honeyd：

命令：honeyd -d -f /usr/local/share/honeyd/honeyd.conf 自己设置的一个空闲 IP 地址

成功启动了 Honeyd 并加载了配置文件 honeyd.conf。Honeyd 版本信息显示为 V1.5c。

程序以 -d（后台运行）和 -f（指定配置文件）选项启动，并监听指定的 IP 地址 192.168.204.22。

上面的这个 windows 模板告诉 honeyd，当一个 client 试图 NMap 探测 honeypot 的指纹时，把它自己伪装成 Linux 2.4.7 (X86)的系统。用 honeyd 自带的 web 脚本虚拟 web 服务。

```
alspp@alspp-virtual-machine:/usr/local/share/honeyd$ sudo arpd 192.168.204.122
arpd[3980]: listening on ens33: arp and (dst 192.168.204.122) and not ether src
00:0c:29:7f:ee:0a
alspp@alspp-virtual-machine:/usr/local/share/honeyd$ sudo honeyd -d -f /usr/loca
l/share/honeyd/honeyd.conf 192.168.204.122
Honeyd V1.5c Copyright (c) 2002-2007 Niels Provos
honeyd[4007]: started with -d -f /usr/local/share/honeyd/honeyd.conf 192.168.204
.122
Warning: Impossible SI range in Class fingerprint "IBM OS/400 V4R2M0"
Warning: Impossible SI range in Class fingerprint "Microsoft Windows NT 4.0 SP3"
honeyd[4007]: listening promiscuously on ens33: (arp or ip proto 47 or (udp and
src port 67 and dst port 68) or (ip and (host 192.168.204.122))) and not ether s
rc 00:0c:29:7f:ee:0a
honeyd[4007]: Demoting process privileges to uid 65534, gid 65534
honeyd[4007]: update_errorcb: failed to get security update information
```

测试机测试 honeyd 主机连通性

```
C:\Users\26032>ping 192.168.204.122

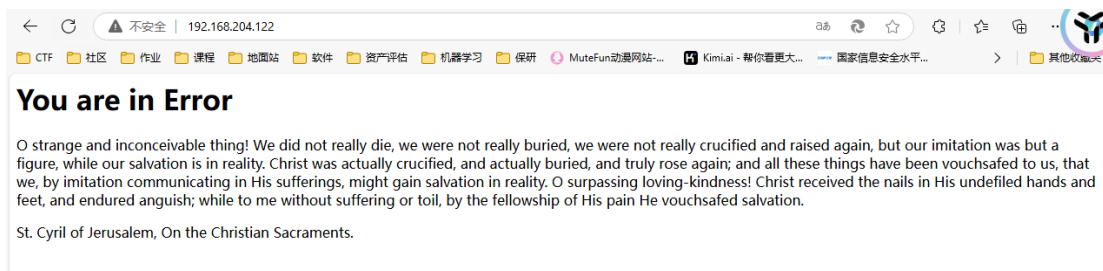
正在 Ping 192.168.204.122 具有 32 字节的数据:
来自 192.168.204.122 的回复: 字节=32 时间=1664ms TTL=255
来自 192.168.204.122 的回复: 字节=32 时间=18ms TTL=255
来自 192.168.204.122 的回复: 字节=32 时间=7ms TTL=255
来自 192.168.204.122 的回复: 字节=32 时间=28ms TTL=255

192.168.204.122 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 7ms, 最长 = 1664ms, 平均 = 429ms
```

回到宿主机，发现捕捉到监听情况：

```
honeyd[15740]: Demoting process privileges to uid 65534, gid 65534
honeyd[15740]: update_errorcb: failed to get security update information
honeyd[15740]: Sending ICMP Echo Reply: 192.168.204.122 -> 192.168.204.171
honeyd[15740]: Sending ICMP Echo Reply: 192.168.204.122 -> 192.168.204.171
honeyd[15740]: Sending ICMP Echo Reply: 192.168.204.122 -> 192.168.204.171
honeyd[15740]: Sending ICMP Echo Reply: 192.168.204.122 -> 192.168.204.171
```

测试机访问 honeyd 模拟的 web 服务（192.168.204.122）



查看命令行中 Honeyd 记录的此次 Web 访问的过程、连接标志和模拟脚本路
径等信息，并记录到实验报告册中。

```
honeyd[15740]: Sending ICMP Echo Reply: 192.168.204.122 -> 192.168.204.171
honeyd[15740]: Connection request: tcp (192.168.204.171:50041 - 192.168.204.122:80)
honeyd[15740]: Connection established: tcp (192.168.204.171:50041 - 192.168.204.122:80) <-> /home/a
lspp/桌面/honeyd/honeyd-1.5c/scripts/web.sh
honeyd[15740]: Connection request: tcp (192.168.204.171:50039 - 192.168.204.122:80)
honeyd[15740]: Connection established: tcp (192.168.204.171:50039 - 192.168.204.122:80) <-> /home/a
lspp/桌面/honeyd/honeyd-1.5c/scripts/web.sh
```

```
honeyd[6493]: update_errorcb: failed to get security update information
honeyd[6493]: Connection request: tcp (192.168.204.171:51620 - 192.168.204.122:80)
honeyd[6493]: Connection established: tcp (192.168.204.171:51620 - 192.168.204.122:80) <->
/home/alspp/桌面/honeyd/honeyd-1.5c/scripts/web.sh
honeyd[6493]: Connection request: tcp (192.168.204.171:51619 - 192.168.204.122:80)
honeyd[6493]: Connection established: tcp (192.168.204.171:51619 - 192.168.204.122:80) <->
/home/alspp/桌面/honeyd/honeyd-1.5c/scripts/web.sh
```

通过 Honeyd 日志输出，以下是此次 Web 访问的相关过程、连接标志和模拟脚本路径等信息：

TCP 连接请求：

日志：`honeyd[6493]: Connection request: tcp (192.168.204.171:51619 - 192.168.204.122:80)`

描述：从 IP 地址 `192.168.204.171` 的端口 `51619` 发起另一个 TCP 连接请求到 IP 地址 `192.168.204.122` 的端口 `80`。

TCP 连接建立：

日志：`honeyd[6493]: Connection established: tcp (192.168.204.171:51619 - 192.168.204.122:80) <-> /home/alspp/桌面/honeyd/honeyd-1.5c/scripts/web.sh`

描述：上述另一个 TCP 连接请求也成功建立，并且同样被重定向到 `web.sh` 脚本处理。

查看模拟脚本的详细内容

《网络安全技术》实验报告

```
alspp@alspp-virtual-machine:/usr/local/share/honeyd$ cat /home/alspp/桌面/honeyd/honeyd-1.5c/scripts/web.sh
#!/bin/sh
REQUEST=""
while read name
do
    LINE='echo "$name" | egrep -i "[a-z:]"'
    if [ -z "$LINE" ]
    then
        break
    fi
    echo "$name" >> /tmp/log
    NEWREQUEST='echo "$name" | grep "GET .scripts.*cmd.exe.*dir.* HTTP/1.0"'
    if [ ! -z "$NEWREQUEST" ] ; then
        REQUEST=$NEWREQUEST
    fi
done

if [ -z "$REQUEST" ] ; then
    cat << _eof_
HTTP/1.1 404 NOT FOUND
Server: Microsoft-IIS/5.0
P3P: CP='ALL IND DSP COR ADM CONO CUR CUSO IVAo IVDo PSA PSD TAI TELo OUR SAMo CNT COM INT NAV ONL PHY PRE PUR UNI'
Content-Location: http://cpmsftwbw27/default.htm
D_Terminal, 04 Apr 2002 06:42:18 GMT
Content-type: text/html
Accept-Ranges: bytes

<html><title>You are in Error</title>
<body>
<h1>You are in Error</h1>
0 strange and inconceivable thing! We did not really die, we were not really buried, we were not really crucified and raised again,
but our imitation was but a figure, while our salvation is in reality. Christ was actually crucified, and actually buried, and truly
rose again; and all these things have been vouchsafed to us, that we, by imitation communicating in His sufferings, might gain salv
ation in reality. O surpassing loving-kindness! Christ received the nails in His undefiled hands and feet, and endured anguish; whil
e to me without suffering or toil, by the fellowship of His pain He vouchsafed salvation.
<p>
St. Cyril of Jerusalem, On the Christian Sacraments.
</body>
</html>
_eof_
    exit 0
fi

DATE=`date`
cat << _eof_
HTTP/1.0 200 OK
Date: $DATE
Server: Microsoft-IIS/5.0
Connection: close
Content-Type: text/plain

Volume in drive C is Webserver
Volume Serial Number is 3421-07F5
Directory of C:\inetpub

01-20-02  3:58a    <DIR>        .
08-21-01  9:12a    <DIR>        ..
08-21-01  11:28a   <DIR>        AdminScripts
08-21-01  6:43p    <DIR>        ftproot
07-09-00  12:04a   <DIR>        iissamples
07-03-00  2:09a    <DIR>        mailroot
07-16-00  3:49p    <DIR>        Scripts
07-09-00  3:10p    <DIR>        webpub
07-16-00  4:43p    <DIR>        wwwroot
           0 file(s)          0 bytes
           20 dir(s)        290,897,920 bytes free

_eof_

```

Nmap 扫描

安装:

```
alspp@alspp-virtual-machine:/usr/local/share/honeyd$ sudo apt install nmap
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
将会同时安装下列软件:
  libblas3 liblinear4 lua-lpeg nmap-common
建议安装:
  liblinear-tools liblinear-dev ncat ndiff zenmap
下列【新】软件包将被安装:
  libblas3 liblinear4 lua-lpeg nmap nmap-common
升级了 0 个软件包，新安装了 5 个软件包，要卸载 0 个软件包，有 18 个软件包未被升级。
需要下载 5,557 kB 的归档。
解压缩后会消耗 26.3 MB 的额外空间。
您希望继续执行吗？ [Y/n] y
获取:1 http://mirrors.tuna.tsinghua.edu.cn/ubuntu focal/main amd64 libblas3 amd64 3.9.0-1build1 [142 kB]
0% [1 libblas3 6,467 B/142 kB 5%] [正在等待报头]
```

扫描 21,22,23,80 端口: nmap -P 192.168.204.122

```
Host is up (0.033s latency).
Not shown: 996 closed tcp ports (conn-refused)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
80/tcp    open  http
```

识别每个开放端口上运行的服务及其版本信息

```
Host is up (0.000045s latency).
All 1000 scanned ports on alspp-virtual-machine (192.168.204.165) are closed
Too many fingerprints match this host to give specific OS details
Network Distance: 0 hops

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.90 seconds
```

操作系统检测:

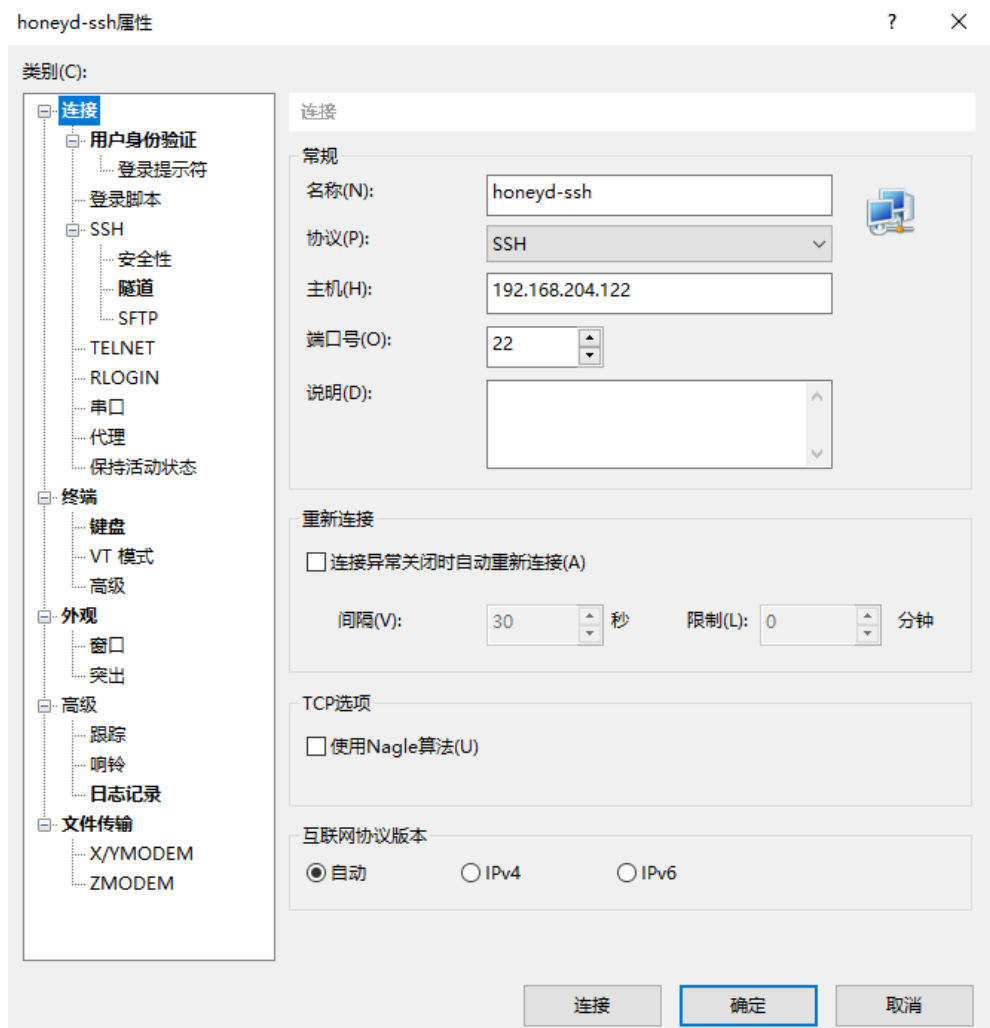
‘Too many fingerprints match this host to give specific OS details’: 由于匹配到的操作系统指纹太多，无法给出具体的操作系统细节。这可能是因为目标主机的响应特征与多种操作系统相似，或者 `nmap` 无法准确识别其操作系统。

4.网络距离:

‘Network Distance: 0 hops’: 网络距离为 0 跳，意味着目标主机与扫描主机在同一网络段内。

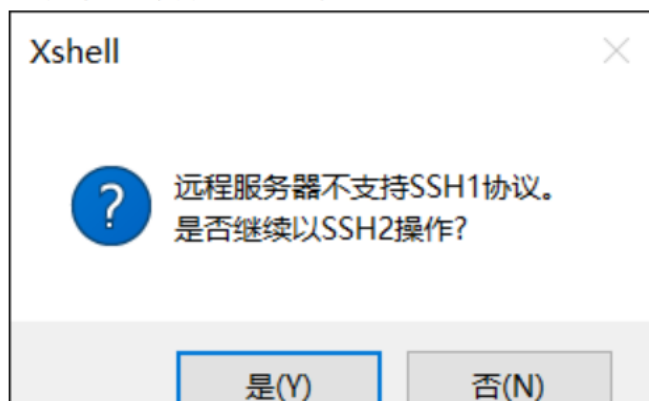
Ssh 连接:

Xshell 连接。

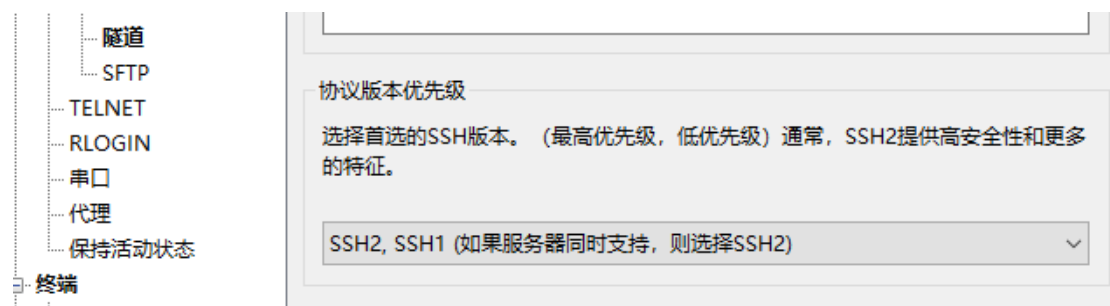


报错:

远程服务器不支持SSH1协议。是否继续以SSH2操作?



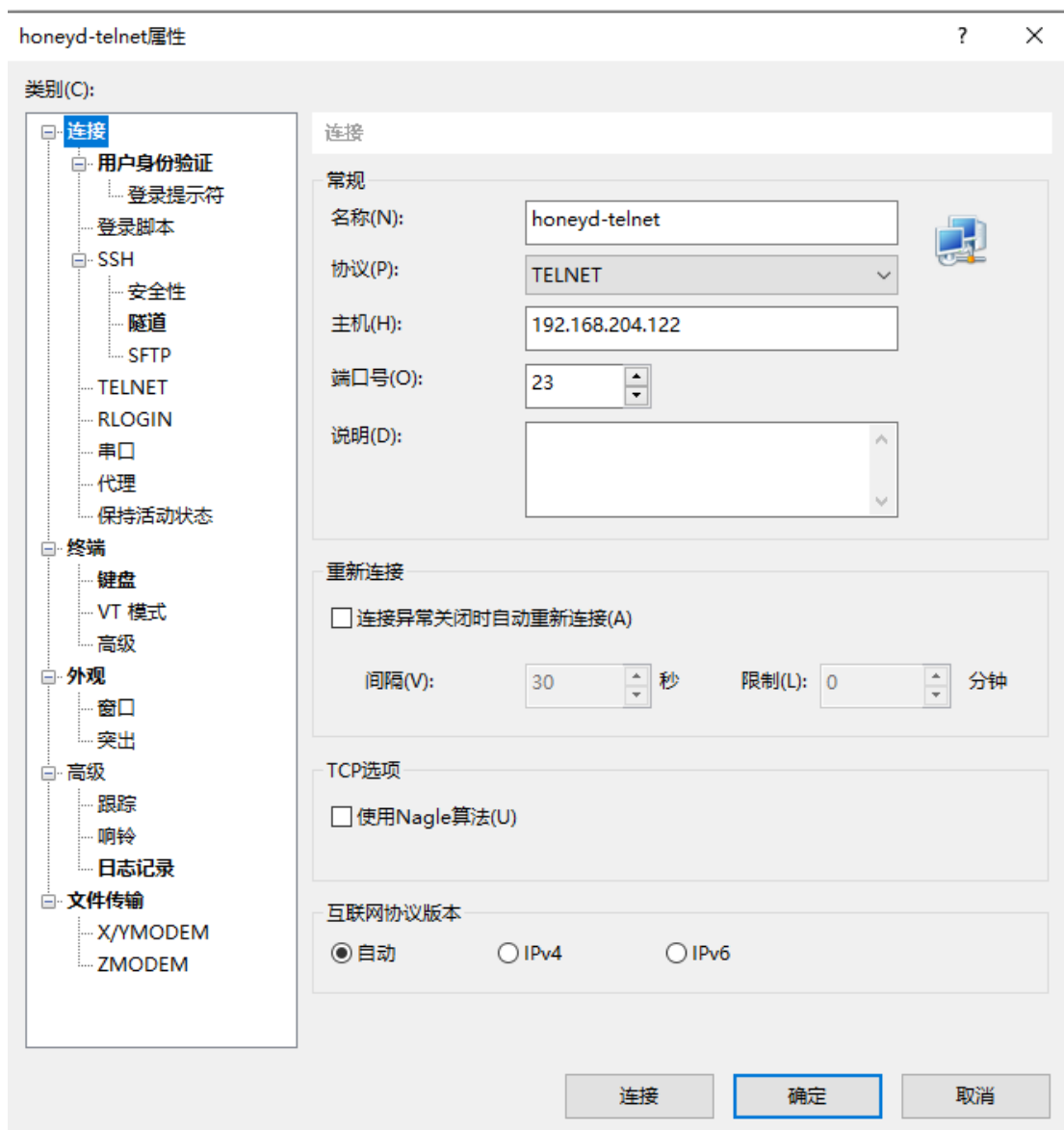
修改 ssh 设置



查看后台监控信息：

```
honeyd[7610]: Connection established: tcp (192.168.204.171:51972 - 192.168.204.122:22) <-> /home/alspp/桌面/honeyd/honeyd-1.5c/scripts/test.sh
honeyd[7610]: Connection closed: tcp (192.168.204.171:51972 - 192.168.204.122:22)
honeyd[7610]: Connection request: tcp (192.168.204.171:51975 - 192.168.204.122:22)
honeyd[7610]: Connection established: tcp (192.168.204.171:51975 - 192.168.204.122:22) <-> /home/alspp/桌面/honeyd/honeyd-1.5c/scripts/test.sh
```

telnet



查看后台监控信息

```
Type 'help' to learn how to use Xshell prompt.
[C:\~]$

Connecting to 192.168.204.122:23...
Connection established.
To escape to local shell, press 'Ctrl+Alt+J'.
Users (authorized or unauthorized) have no explicit or
implicit expectation of privacy. Any or all uses of this
system may be intercepted, monitored, recorded, copied,
audited, inspected, and disclosed to authorized site,
and law enforcement personnel, as well as to authorized
officials of other agencies, both domestic and foreign.
By using this system, the user consents to such
interception, monitoring, recording, copying, auditing,
inspection, and disclosure at the discretion of authorized
site.

Unauthorized or improper use of this system may result in
administrative disciplinary action and civil and criminal
penalties. By continuing to use this system you indicate
your awareness of and consent to these terms and conditions
of use. LOG OFF IMMEDIATELY if you do not agree to the
conditions stated in this warning.

User Access Verification

Username:asd
Password:
% Access denied

Username:
% Username: timeout expired!

Username: admin
Password:
% Access denied
Connection closing...Socket close.

Connection closed by foreign host.

Disconnected from remote host(honeyd-telnet) at 20:19:13.

Type 'help' to learn how to use Xshell prompt.
[C:\~]$
```

```
71:68)
honeyd[7610]: Connection request: tcp (192.168.204.171:52002 - 192.168.204.122:23)
honeyd[7610]: Connection established: tcp (192.168.204.171:52002 - 192.168.204.122:23) <-> /home/alspp/桌面/honeyd/honeyd-1.5c/scripts/router-telnet.pl
honeyd[7610]: E(192.168.204.171:52002 - 192.168.204.122:23): Attempted login:asd/123
honeyd[7610]: E(192.168.204.171:52002 - 192.168.204.122:23): Attempted login:admin/123456
honeyd[7610]: Expiring TCP (192.168.204.171:52002 - 192.168.204.122:23) (0x5592b52eb310) in state 7
honeyd[7610]: Expiring OS fingerprint for 192.168.204.171
```

四、讨论与分析

如何利用 Honeyd 实现跨网段的模拟？参考学习网络配置实例。

通过创建多个虚拟网络和相应的虚拟主机来实现。

在 Honeyd 配置文件中，使用 `def net` 指令定义多个不同的网络段。如何对于每个网络段，使用 `def host` 指令定义虚拟主机，并指定属于哪个网络。在配置文件中定义虚拟路由器，并设置路由规则，使得不同网络段可以互相通信。使用配置文件启动 Honeyd，加载上述定义的网络和主机配置。

● Honeyd 如何实现虚拟路由器的模拟？

1. **定义路由器：** 使用 `def router` 指令在 Honeyd 配置文件中定义一个虚拟路由器。
2. **配置接口：** 为虚拟路由器定义接口，并指定连接到哪些网络。
3. **设置路由规则：** 配置路由规则，使得虚拟路由器可以转发不同网络之间的流量。
4. **启动 Honeyd：** 使用配置文件启动 Honeyd，加载虚拟路由器和路由规则。

例如：

```
route entry 172.31.0.100 network 172.31.0.0/16
route 172.31.0.100 link 172.31.0.0/24

route 172.31.0.100 add net 172.31.1.0/24 172.31.1.100
route 172.31.1.100 link 172.31.1.0/24

create windows
set windows personality "Microsoft Windows NT 4.0 SP3"
set windows default tcp action reset
set windows tcp port 80 open
set windows tcp port 25 open
set windows tcp port 21 open

create router
set router personality "Cisco 7206 running IOS 11.1(24)"
set router default tcp action reset
add router tcp port 23 "Script/router-telnet.pl"
```

```
bind 172.31.0.100 router
bind 172.31.1.100 router
```

● 实验步骤 5 中启动 Honeyd 的第一种方法和第 2 种有什么区别？还有哪些参数可以使用？

`honeyd -d -f /usr/local/share/honeyd/honeyd.conf` 自己设置的一个空闲 IP 地址

- `-d`：这个参数让 Honeyd 在后台运行，即以守护进程模式启动。
- `-f`：这个参数后面跟配置文件的路径，指定 Honeyd 使用哪个配置文件来启动。
- `<自己设置的一个空闲 IP 地址>`：这是指定 Honeyd 监听的 IP 地址，这个地址应该是网络中未被使用的，以便 Honeyd 可以模拟网络流量。

```
honeyd -d -f /usr/local/share/honeyd/honeyd.conf 192.168.117.251 -a
/usr/local/share/honeyd/nmap.assoc 自己设置的一个空闲 IP
```

- -d 和 -f 的含义与第一种方法相同。
- <自己设置的一个空闲 IP>: 这同样是指定 Honeyd 监听的 IP 地址。
- -a: 这个参数后面跟一个 .assoc 文件的路径, 这个文件包含了关联信息, 用于 Honeyd 的关联引擎。关联引擎可以帮助 Honeyd 理解网络流量中的不同部分是如何关联的, 比如一个 TCP 三次握手过程。

区别:

两种方法的主要区别在于是否使用 -a 参数来指定 .assoc 文件。这个文件对于 Honeyd 的高级功能是重要的, 特别是在处理复杂的网络流量和模拟复杂的网络服务时。

其他可用参数:

除了您已经使用的参数外, Honeyd 还支持其他一些参数, 包括但不限于:

- -p <端口>: 指定 Honeyd 监听的端口号。
- -i <接口>: 指定 Honeyd 绑定到的网络接口。
- -u <用户>: 以指定的用户身份运行 Honeyd。
- -g <组>: 以指定的组身份运行 Honeyd。
- -D: 启用调试模式, 提供更多的输出信息。
- -n: 不启动任何活动代理。
- -r <脚本根目录>: 指定存放 Honeyd 脚本的目录。
- -x: 以原始模式运行, 不解析 .assoc 文件。
- -v: 显示版本信息。
- -h: 显示帮助信息。

- 你还能用 honeyd 虚拟出哪些服务? 其配置文件应该如何编写。

使用 Honeyd 可以虚拟出各种常见服务, 包括 HTTP、SMTP、FTP、Telnet、DNS、以及其他自定义 TCP/UDP 服务。每个服务的虚拟化可以通过定义端口响应和关联服务脚本来实现

1. HTTP 服务:

模拟 HTTP 服务, 使虚拟蜜罐看起来像一个 Web 服务器。

例:

```
create web_server
set web_server personality "Apache httpd 2.2.21"
set web_server default tcp action reset
add web_server tcp port 80 "sh /scripts/http.sh"
bind 192.168.1.10 web_server
```

2. SMTP 服务

使蜜罐模拟 SMTP 电子邮件服务器。

例:

```
create mail_server
set mail_server personality "Postfix"
set mail_server default tcp action reset
add mail_server tcp port 25 "sh /scripts/smtp.sh"
bind 192.168.1.11 mail_server
```

3. FTP 服务

模拟 FTP 文件传输服务。

例：

```
create ftp_server
set ftp_server personality "vsftpd 3.0.3"
set ftp_server default tcp action reset
add ftp_server tcp port 21 "sh /scripts/ftp.sh"
bind 192.168.1.12 ftp_server
```

4. Telnet 服务

模拟远程登录服务，使虚拟蜜罐模拟 Telnet。

例：

```
create telnet_server
set telnet_server personality "Cisco 7206"
set telnet_server default tcp action reset
add telnet_server tcp port 23 "perl /scripts/telnet.pl"
bind 192.168.1.13 telnet_server
```

5. DNS 服务

模拟 DNS 服务，通常用于检测 DNS 查询请求。

例：

```
create dns_server
set dns_server personality "BIND 9.8.2"
set dns_server default udp action reset
add dns_server udp port 53 "sh /scripts/dns.sh"
bind 192.168.1.14 dns_server
```

五、实验者自评（从实验设计、实验过程、对实验知识点的理解上给出客观公正的自我评价）

1.实验设计

本次实验的设计目标明确、内容丰富，从 Honeyd 的安装、配置到实际应用进行了较为全面的覆盖，确保了实验的系统性和完整性。在设计中，实验者合理地分配了不同模块的任务，包括安装依赖库、配置文件编写、端口模拟等步骤，均体现了对蜜罐技术整体结构的深入思考。实验设计考虑到网络协议和服务的模拟，并加入了网络扫描、服务访问等测试环节，有助于检验 Honeyd 的模拟效果。若在未来的实验中加入一些高交互蜜罐的学习和对比，将使实验内容更具广度和深度。

2.实验过程

在实验过程中，实验者细致地完成了 Honeyd 的安装和配置操作。面对安装过程中遇到的依赖性问题，实验者通过文档查找和命令调试，成功解决了依赖库缺失等问题，保障了

Honeyd 的顺利安装。这一过程提高了实验者在网络环境搭建中的问题解决能力，并积累了故障排查经验。在实际服务模拟中，实验者通过对多种协议端口的配置和验证，准确记录并分析了每次访问和连接的情况，验证了 Honeyd 的基本功能和网络行为捕捉能力，确保了实验结果的准确性和可靠性。

3.对实验知识点的理解

通过本次实验，实验者对低交互蜜罐的工作原理、特点和适用场景有了更为深入的理解，尤其是在网络安全防护中的作用、应用优势和局限性方面有了清晰的认知。配置文件的编写环节帮助实验者加深了对服务模拟的理解，提升了对不同协议、端口及操作系统模拟的认识。同时，通过对 Nmap 扫描结果的分析，实验者加深了对网络扫描和服务识别的理解，并进一步掌握了识别蜜罐的基础知识。

六、附录：关键代码（给出适当注释，可读性高）

<https://www.freesion.com/article/27571315419/>

<https://blog.51cto.com/fergusj/1567799>

<https://blog.csdn.net/accepthjp/article/details/46399715>